Autonomous Car: Group 2

*A prototyping paper

Arya Karimi 2219954 | Moiz Zaheer Malik 2220074 | Akinwande Mofifoluwa Ipadeola 2220059 | Qaiser Mustafa 2220058

I. Introduction

Prototyping is an essential tool for engineers and has been recognized as part of the product development process worldwide[5]. For many students new to the world of software programming and embedded systems, there is an urge to dive straight into the programming and to ignore customs that have been outlined and fine-tuned over decades that lead not only to better products but also make the lives of everyone involved in the product development much easier. Our tasks for this project were to abide by these principles while making a line-following autonomous vehicle, that could detect obstacles and perform certain actions based on the color of the obstacle.

II. DESIGN

A. REQUIREMENTS

Requirements engineering helps address the questions of which problems should be solved, how to solve them, and who is responsible for solving them[15]. Following good requirement engineering practices helps achieve the primary objective of making sure the final product matches the initial needs[4]. The requirements the car had to meet were Detection, Speed control, and Parking. These requirements were classified as Functional and Non-functional requirements and were modeled by the aid of a requirement diagram.

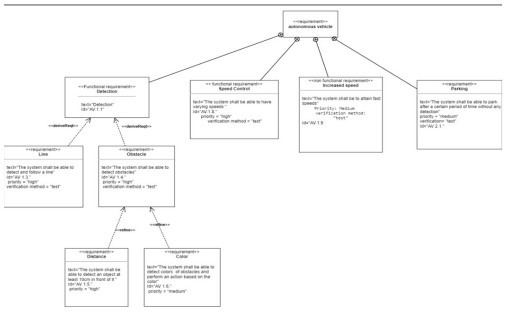


Fig. 1. Requirement diagram

Further sub-requirements like Obstacle detection and Line detection were also specified.

B. BLOCK DIAGRAM AND COMPONENTS

Block diagrams are graphical representations of a system, showing how the major components or functions are interconnected. This form of diagram uses blocks to represent system components and lines to show the relationships and flow of information between them. Block diagrams help with the visualization of the overall structure of the system. Furthermore, within this diagram, one can find clear ways to communicate complex system interactions to the consumer.

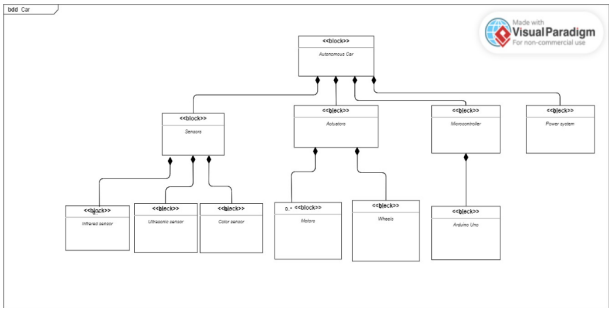
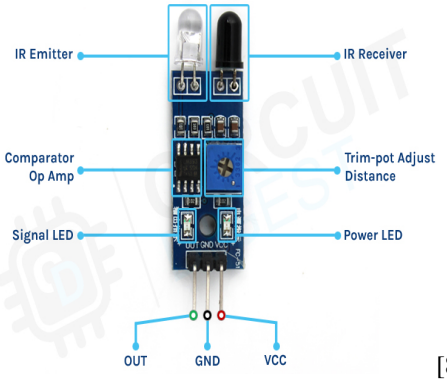


Fig. 2. Block diagram

After specifying the requirements and through the aid of the block diagram, we identified the necessary components for an operational car. These components included infrared sensors for line detection, an ultrasonic sensor for obstacle detection, a color sensor to assist with color detection, motors for various movements, and a motor driver. To serve as the central control unit for all these components, we employed the Arduino Uno microcontroller.

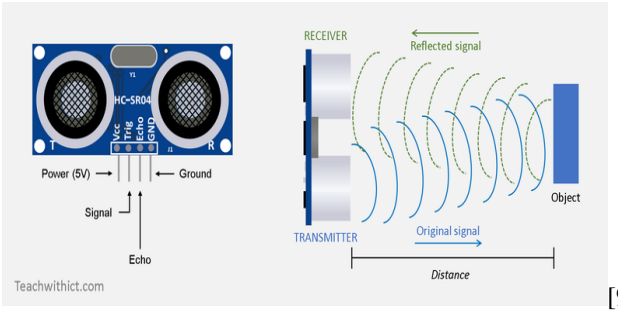
1) INFRARED SENSOR: IR sensor is a radiation-sensitive optoelectric component with spectral sensitivity in the infrared range of 780nm - 50m [3]. It consists of two parts, the emitter circuit and the receiver circuit. The emitter is an IR LED and the detector is an IR photodiode. The IR photodiode is sensitive to the IR light emitted by an IR LED. The photodiode's resistance and output voltage change proportionately to the received IR light. This is the underlying working principle of the IR sensor [4]. A typical IR sensor module consists of an IR Reciever LED that detects the Infrared signals, an amplifier that amplifies the received signal, and a demodulator circuit that receives and processes the signal [5]. Other components are the Distance Adjust, Power LED, and Obstacle LED.



[8]

Fig. 3. Infrared Sensor

2) ULTRASONIC SENSOR: The ultrasonic sensor is one of the most reliable forms of detecting proximity. It is an instrument that measures distance using ultrasonic sound waves. Using a transducer, the ultrasonic sensor is able to send and receive ultrasonic pulses that bounce off of an object, the sound waves that are emitted by the sensor are at a frequency that is above the range of human hearing, one of the transducers works as a transmitter and the other functions as a receiver, almost like a microphone. The sensor then determines the distance to the obstacle by measuring the time it takes between sending and receiving the pulses.[2] The formula for the distance calculation is as follows: Distance = [(Time)x(Speed of sound)]/2. Sound travels at approximately 340 m/s, or 340 x 100 cm/s, dividing the whole equation by 2 is due to the fact that the sound waves must travel to the object, and back to the receiver.



[9]

Fig. 4. Ultrasonic Sensor

3) COLOR SENSOR: Color sensors detect RGB values by receiving ambient light using a photo-diode[6]. To detect the color of material, three main types of equipment are required. A light source to illuminate the material surface, a surface whose color has to be detected and the receivers which can measure the reflected wavelengths. Three filters with wavelength sensitivities of 580nm, 540nm and 450nm are used to measure the wavelength of red, green and blue colors respectively[7].



[1]

Fig. 5. Color Sensor

4) MOTOR DRIVER: The L298N module is a high voltage, high current dual full-bridge motor driver module for DC and stepper motor control.Two methods are used by this module to regulate the rotational direction and speed of DC motors. These are H-Bridge (for rotation direction control) and PWM (for speed control). One stepper motor or two DC motors may be controlled simultaneously using these modules. It consists of four Output pins which control the direction of the motors and two enable pins that control the speed of the motors.

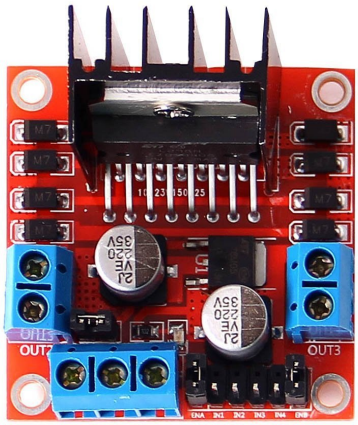


Fig. 6. Motor driver

5) MOTORS: A DC motor is an electrical motor that uses direct current (DC) to produce mechanical force[16]. The permanent magnet DC motor (PMDC) is the most commonly used type of small DC motor available producing a continuous rotational speed that can be easily controlled. A DC motor consists basically of two parts, the stationary body of the motor called the ”Stator” and the inner part which rotates producing the movement called the ”Rotor”[14].



[13]

Fig. 7. DC Motor

6) ARDUINO UNO: The Arduino Uno is a microcontroller board based on the ATmega328p. It has 14 digital input/output pins, and 6 analog inputs. It is very popular due to its simplicity, versatility and the many resources available. It is also very accessible for beginners and flexible for advanced users.



[2]

Fig. 8. Arduino Uno board

C. PIN CONFIGURATION

1) Motor Driver: Pins for Motor driver

L298N MODULE	PIN CONFIGURATION
EN_A	3
EN_B	11
IN_1	4
IN_2	5
IN_3	6
IN_4	7

2) Infrared Sensor: Pins for Infrared Sensor

IR SENSOR	PIN CONFIGURATION
IR_PIN_1	12
IR_PIN_2	8

3) Color sensor: Pins for color sensor

COLOR SENSOR	PIN CONFIGURATION
S0_PIN	A1
S1_PIN	A2
S2_PIN	A3
S3_PIN	A4
OUT_ENABLE	10

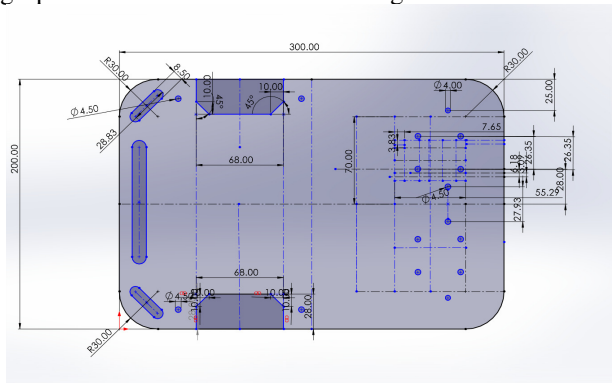
4) Ultrasonic sensor: Pins for Ultrasonic sensor

Ultrasonic	PIN CONFIGURATION
ECHO_PIN	2
TRIG_PIN	11

D. MAIN DESIGN

For the design of our car, we used various tools and techniques which will be explained in depth. Notably, the software used to design and construct all of our components was Solidworks, a tool that helps with computer aided diagrams.

1) CHASSIS: The top and bottom chassis were designed meticulously to fit all components. It measures 20cm in width and 30cm in length. Plywood was selected for the chassis due to its durability, strength, and ease of machining. To cut all the holes in the chassis, a technique known as laser wood cutting was employed. It is a precision technique that uses high-powered laser beam to cut or engrave wood materials.



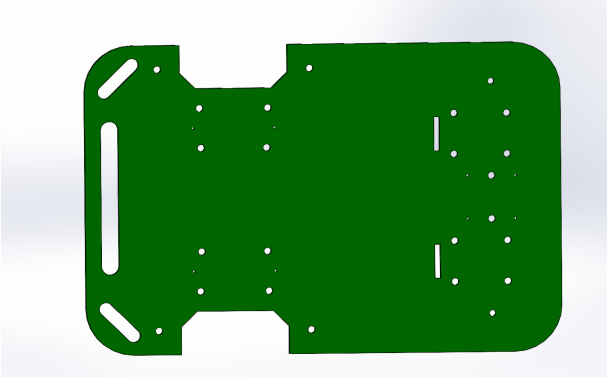


Fig. 9. Chassis of the car and its dimensions

2) COMPONENT HOLDERS: Specific holders were designed for each component so that they could be assembled

uniformly on the car. To bring the holders to life, we employed a technique known as 3D printing or additive manufacturing. This technology enables precise part production which is crucial for high-level prototyping and manufacturing. However, as this technique is time-consuming and demands more materials, we had to be attentive to the size, and importance of the component we wanted to 3D print.[3]

- Motor holder: To maintain stability and alignment, a motor holder was designed. Each motor holder featured 10mm long, 4.5mm diameter holes with screw threads at each corner, allowing them to be securely fastened to the chassis from the bottom.

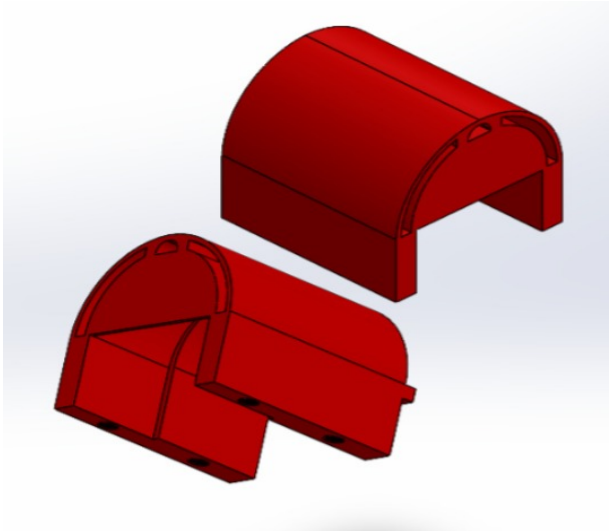


Fig. 10. Motor holder designed on Solidworks

- IR sensor holder: Two IR sensor holders were designed to fit the specific dimensions of the IR sensors. Each holder had screw holes at each corner, ensuring they could be securely fastened to the bottom of the chassis. The holders were designed to position the sensors accurately for optimal line detection.

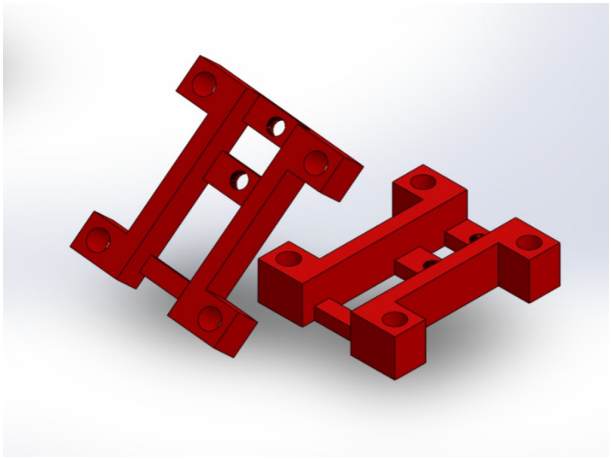


Fig. 11. Infrared Sensor holder designed on Solidworks

- Third wheel holder: The small third wheel required a custom holder due to its size and lack of screw holes. The holder was designed with a spring mechanism to provide suspension and prevent the wheel from dislodging. This design included two screw holes to attach it securely to the chassis. The spring mechanism functioned as a suspension system, absorbing shocks and maintaining stability during movement.

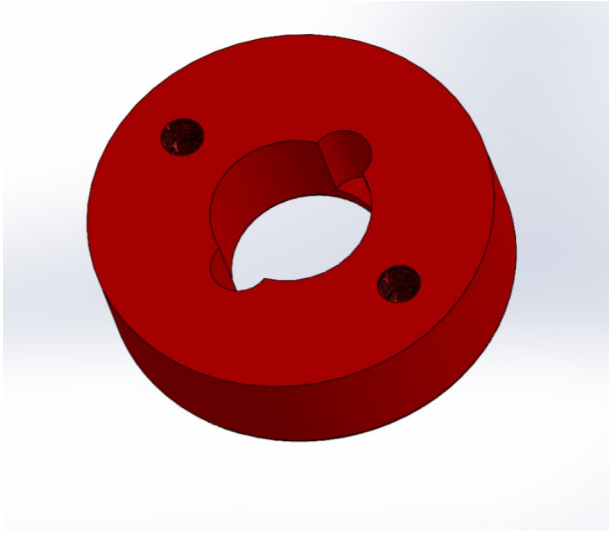


Fig. 12. Third wheel holder designed on Solidworks

- Ultrasonic sensor mount: Initially, we designed a servo holder to allow the ultrasonic sensor to rotate and take measurements from the sides. This involved creating a secure mount for the servo motor, which would then be attached to the ultrasonic sensor. However, after further consideration, we opted for a fixed cover for the ultrasonic sensor, which was mounted on top of the car. This decision simplified the design and reduced potential points of failure.

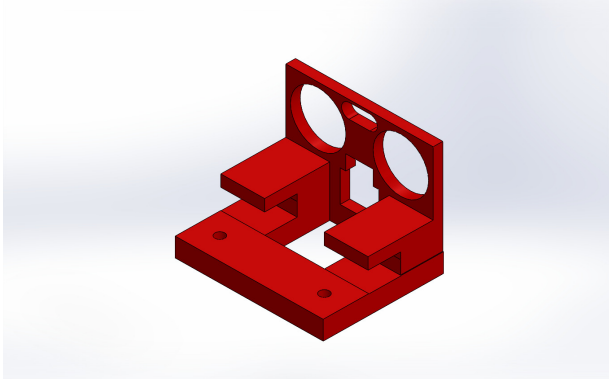


Fig. 13. Ultrasonic mount designed on Solidworks

3) SIDES: To cover the sides of the car, we employed a wood bending technique. The particular one used was Kerf bending, which involves making a series of slits (kerfs) in

the wood to make it flexible enough to bend[12]. The kerf pattern includes slits that allow the wood panel to flex and bend smoothly. A 514mm long wooden sheet was kerf-cut to cover the sides. This sheet was designed with six teeth for better positioning and stability. The teeth fit into airtight holes on the top part of the chassis, ensuring a secure and aligned fit.

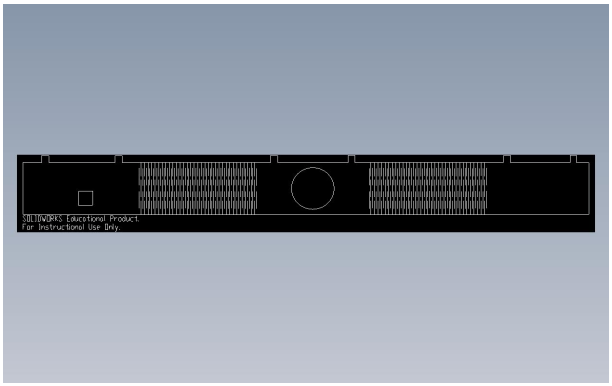


Fig. 14. Sides of the car designed on Solidworks

4) FINAL DESIGN: After attaching all the components, we secured the bottom part of the chassis to the top part to ensure stability. This involved aligning all screw holes and ensuring that all components were tightly fastened. A laser-cut back wall was added to cover the rear side of the car, providing additional protection and structural integrity. The back wall was precisely cut to match the dimensions of the car and included mounting points for additional components if needed.

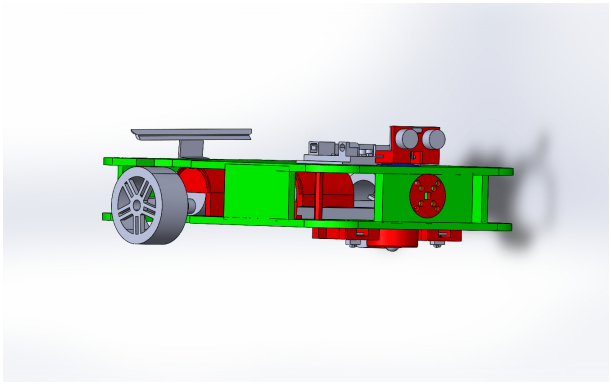


Fig. 15. 3D view of the Solidworks model

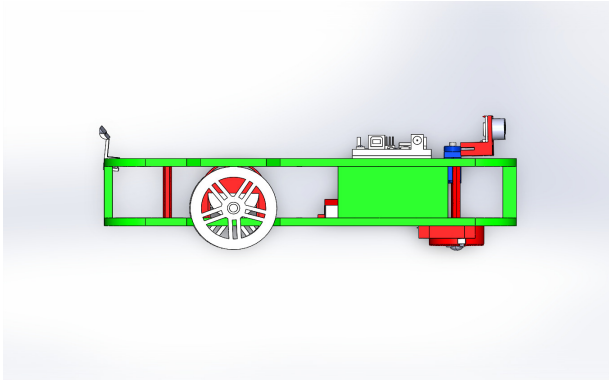


Fig. 16. Side view of the Solidworks model

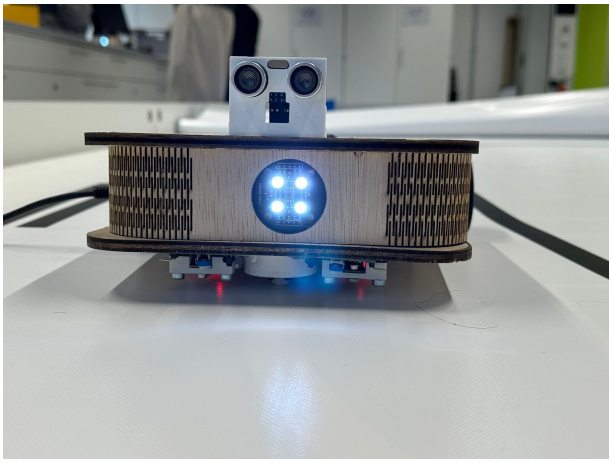


Fig. 17. Front view of the fully assembled car

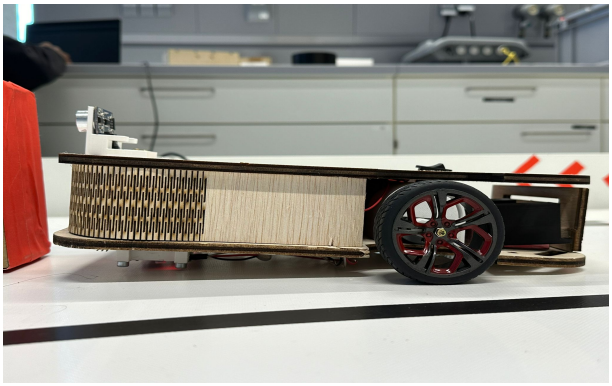


Fig. 18. Side view of the fully assembled car

III. SOFTWARE

A. ACTIVITY DIAGRAM

The major diagram used to describe an activity is called an activity diagram. It describes and defines the actions in an activity along with the flow of input/output and control between them[11]. This kind of diagram also provides a detailed view of how the system should function. Furthermore,

activity diagrams are a great communication tool between the developers and the user, this is due to activity diagrams being able to present complex processes in a visual format. The benefits of activity diagrams also include simple error detection, efficiency, and as previously mentioned, communication and alignment. For our project, we found that an activity diagram was needed as it was important to show how the flow and functions of the car.

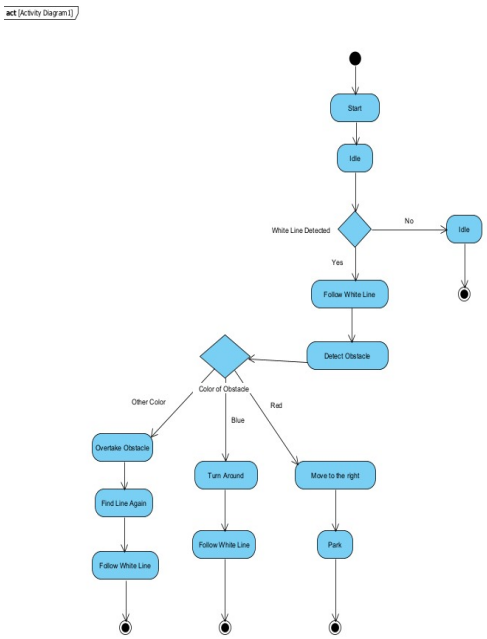


Fig. 19. Activity diagram for the car

For our activity diagram, we aimed for simplicity and efficiency, as illustrated by the diagram above, the initial node marks the start of the workflow, and then the solid arrows represent the control flow. Here we see that after the car starts it moves into the idle activity node until it detects a white line. When the car detects the white line, it follows it until the ultrasonic sensor detects an obstacle. The decision node is for the color of the obstacle, if the color of the obstacle is blue, the car turns around and follows the line, if it's red, it moves to the right and parks. Finally, if the obstacle color is another color than red and blue then the car will overtake the obstacle and find the line again.

B. STATES AND TIMED AUTOMATA

A state diagram essentially is a state machine, consisting of states, transitions, events, and activities. While the activity diagram shows a flow of control from activity to activity across a number of objects involved in the execution of those activities, the state diagram shows the flow of control from state to state within a single object[10]. During software development, we identified the need for a state diagram to enhance code clarity and execution speed. The car's requirements led to the derivation of three main states: Idle, Line Follower, and Detection. In the idle state, the car is completely inactive and only upon detecting a line does it go into the line follower state. If the car comes across an obstacle during its line follower state, then the detection state is triggered and a set of actions are performed dependent on the color of the obstacles. A state chart was used to help model these various states and their transitions.

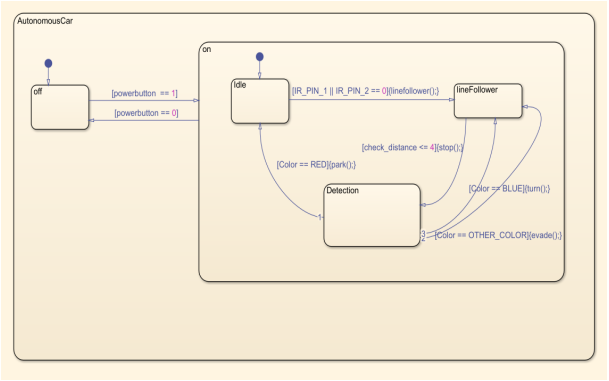


Fig. 20. State chart showing different states of the car with transitions

For timed automata, UPPAAL, a tool for modeling, validation and verification of real-time systems, was used. After creating the main and sub-templates, synchronizing, and simulating, there was no deadlock present, and the temporal properties were also verified.

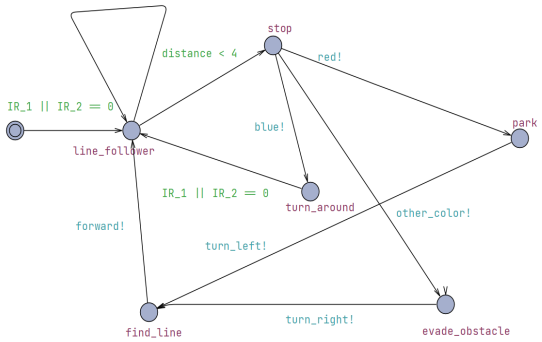


Fig. 21. State machine modeled on UPPAAL

C. CODE

The code and algorithm were written in C++, an object-oriented programming language. The choice to write in C++ was simply to challenge ourselves, but there were also other benefits like function overloading which C does not support. While writing the code, we ensured to abide by correct software engineering practices like using meaningful and understandable names for functions and variables, using correct data types, and creating header files for all components used. All of these helped with writing clear, concise, and easily modifiable code.

1) HEADER FILES: As mentioned earlier, we created header (.h) and source (.cpp) files for each component. Below n figure 20 is an image of the Motor class in the header file. This class includes function declarations with parameters (when needed) and pin initializations required for motor functionality.

```
#ifndef MOTOR_H
#define MOTOR_H

#include "Arduino.h"

class Motor{
  public:
    void motor_init();
    void move_forward(int);
    void move_backward(int);
    void move_forward_left(int);
    void move_backward_left(int);
    void move_forward_right(int);
    void move_backward_right(int);
    void stop();
    void stop_left();
    void stop_right();
    void left(int);
    void right(int);
   static const int EN_A = 3;
   static const int EN_B = 11;
   static const int MOTOR_PIN_1 = 4;
   static const int MOTOR_PIN_2 = 5;
   static const int MOTOR_PIN_3 = 6;
   static const int MOTOR_PIN_4 = 7;

};

#endif
```

Fig. 22. Motor header file and all its functions

An example for a function definition is shown below for the function "move_forward_left" which controls the left motor by moving it forward at a specified speed.

```
void Motor::move_forward_left(int speed){
    analogWrite(EN_A, speed);
    digitalWrite(MOTOR_PIN_3, LOW);
    digitalWrite(MOTOR_PIN_4, HIGH);
}
```

Fig. 23. A motor function definition

2) MAIN FUNCTIONS: Many functions were defined for the proper operation of the car. These are some of the important ones.

- distance(): This function returns an Integer value, and it calculates the distance of an obstacle away from the ultrasonic sensor by multiplying the time it takes for the ultrasonic waves to travel and return to the sensor by the speed of sound divided by 2.

```
int Ultrasonic::distance(){
    int counter = 0;
    int distance = 0;
    counter = pulseIn(ECHO_PIN, HIGH);
    distance = counter * 0.017;
    return distance;
}
```

Fig. 24. A function to calculate distance

- Park(): In this function, the motor and consequently the car moves backward, and then to the right and parks there.

```
void Car::park(){
  motors.move_backward(90);
  delay(1000);
  motors.right(90);
  delay(1000);
  motors.move_forward(90);
  delay(1000);
  motors.left(90);
  delay(1000);
  motors.stop();
  delay(5000);
  motors.move_forward(90);
  delay(1000);
  motors.left(90);
  delay(500);
  motors.stop();
  delay(1000);
  }
```

Fig. 25. A function to park the car

- Turn(): In this function, the left motor moves forward, while the right motor moves backward for a specified duration. This coordinated action causes the car to rotate around its axis, completing a precise 180-degree turn.

```
void Car::turn(){
 motors.move_backward_left(100);
 motors.move_forward_right(100);
 delay(1700);
 motors.stop();
 delay(500);
 }
```

Fig. 26. A function to turn the car

- `line_follower()`: In this function, two other functions are called, the `run_right` and `run_left` functions of the Car class. In these functions, a simple conditional algorithm is employed: if the IR sensors get inactive (indicating that the car is on the black part of the track), the motors stop. Otherwise, they move forward at a specified speed.

```
void line_follower(){
    car.run_right();
    car.run_left();
  }
```

```
void Car::run_left(){
 if(digitalRead(IR_PIN_1) == HIGH){
       motors.stop_left();
   }
  else{
      motors.move_forward_left(255);
    }
}

void Car::run_right(){
 if(digitalRead(IR_PIN_2) == HIGH){
       motors.stop_right();
   }
  else{
      motors.move_forward_right(255);
    }
}
```

Fig. 27. Function for line following and its sub functions

- `color_detection()`: In this function, once the necessary values from the color sensor have been obtained and calibrated, the corresponding color variables are stored in an enum variable called 'COLORS.' This process relies on conditional statements.

```
void color_detection(){
  int red_color = color.red();
  int blue_color = color.blue();
  int other_color = color.green();

  if(red_color > blue_color && red_color > other_color){
    COLOR = RED;
  }
  else if(blue_color > red_color && blue_color > other_color){
    COLOR = BLUE;
    }
  else{
    COLOR = OTHER_COLOR;
    }
    delay(1000);
  }
```

Fig. 28. Function for color detection

3) MAIN LOOP: State machines, as mentioned earlier, contribute to faster executable code. A common approach to implementing state machines is through switch-case statements. To define the states, we created an enum variable called 'STATE,' which encompasses all the states used within the loop. The sequence of the main loop was derived from the earlier created activity diagram. Here's how the state transitions work for the car:

1) Idle State: Initially, the car is in the Idle state.
2) Line Follower State: If a specific flag is triggered, the car transitions to the line follower state.
3) Detection State: To enter the detection state, the car must be within 4 cm of an obstacle. Here, the color sensor checks the obstacle's color: If the obstacle is red, the car parks. If it's blue, the car performs a turn-around maneuver on the line. For any other color, the car evades the obstacle by maneuvering around it.

```
void loop() {
  STATE = IDLE;
  flagg = car.check_flag();
  Serial.println(flagg);
  if(flagg == 1){
    STATE = LINE_FOLLOWER;
    }

  ultrasonic.trigger();
  check_distance = ultrasonic.distance();

 if(check_distance <= 4){
    car.stop();
    STATE = DETECTION;
    color_detection();
    }

  switch(STATE){
    case DETECTION:
      switch(COLOR){
        case RED:
            car.park();
            delay(5000);
            break;
        case BLUE:
            car.turn();
            delay(2000);
            break;
        case OTHER_COLOR:
          car.evade();
          delay(1000);


  case LINE_FOLLOWER:
    line_follower();
    break;

  case IDLE:
     car.stop();
     ultrasonic.low();
     break;
 }
```

Fig. 29. Main loop

IV. CONCLUSION

In this paper, we have detailed our strategy and the practical implementation of our ideas, from systems design to prototyping, woodworking, and programming, for our autonomous line-following car. During the design phase we implemented traditional woodworking techniques and modern 3D printing methods which allowed us to create a functional and stable prototype for our line-following car. The use of kerf bending provided the necessary flexibility for the chassis design, while the custom 3D printed parts ensured precise fitting and functionality of the various electronic components. The final design met our objectives, providing a solid foundation for further development and testing. During the software design phase, we adhered to established software engineering principles, while also using System engineering model tools like activity diagrams as guidance which in turn improved our critical thinking skills as we navigated the complexities systems and programming. By balancing the excitement of hands-on crafting, with theoretical knowledge gained in the previous semesters, we not only achieved our project goals but also gained valuable insights into the importance of well balanced prototyping in engineering and embedded systems. It was important to remain focused throughout this course to be able to complete all the tasks given effectively with strong team-work, cohesion, and responsibility.

V. APPENDIX: TASKS AND COMMITMENTS

A. *Moiz Zaheer Malik*

Responsibilities: Main Design of Car Body: Moiz led the conceptualization and creation of the car's overall structure. This involved designing the chassis, ensuring it could house all necessary components, and making sure it was aerodynamically sound and structurally robust.

Motor Holder: He designed the holder for the motor, ensuring it was securely mounted and efficiently transferred power to the wheels.

Motor Library: Moiz developed a software library to interface with and control the car's motor. This included functions for starting, stopping, and adjusting the motor's speed.

Requirement Diagram: He created the requirement diagram to outline and clarify the project's functional and non-functional requirements, serving as a roadmap for development.

Side Cover: Moiz also created the side cover to hide the inner components of the car by using a wood bending method. This ensured the internal components were protected and the car had a cleaner, more aesthetic appearance (25 percent)

B. *Arya Karimi*

Responsibilities: Ultrasonic Holder: Arya designed the holder for the ultrasonic sensors, ensuring they were positioned correctly to detect obstacles effectively.

Ultrasonic Library: He developed a software library to manage the ultrasonic sensors, including functions to measure distances and detect obstacles.

Activity Diagram: Arya created the activity diagram, illustrating the workflow of the car's functionalities and how different actions were triggered by sensor inputs.(25 percent)

C. *Akinwande Mofifoluwa*

Responsibilities: IR Holder: Mofifoluwa designed the holder for the infrared sensors, ensuring they were accurately placed to detect lines on the ground.

Car Library: He developed a comprehensive software library for the car, integrating various sensor inputs and controlling outputs.

Main .INO File: Mofifoluwa was responsible for the main .INO file in the Arduino IDE, which served as the primary code file to control the car's operations.

State Machine Diagram: He created the state machine diagram to model the car's behavior in different states and transitions based on sensor inputs.(25 percent)

D. *Qaiser Mustafa*

Responsibilities: 3rd Wheel Holder: Mustafa designed the holder for the car's third wheel, ensuring stability and proper alignment.

Color Library: He developed a software library to process inputs from the color sensor, allowing the car to perform actions based on detected colors.

Block Definition Diagram: Mustafa created the block definition diagram, detailing the system's components and their interactions.(25 percent)

REFERENCES

[1] URL: https://joy-it.net/en/products/SEN-Color. (accessed 28.06.2024).

[2] URL: https://www.amazon.de/Arduino-A000066-UNO-Rev3/dp/B008GRTSV6. (accessed 28.06.2024).

[3] URL: https://3dprinting.com/what-is-3d-printing/. (accessed 28.06.2024).

[4] Kim Fowler Chris Hersman. "Mission-Critical and Safety-Critical Systems Handbook". In: Newnes, 2010. Chap. 5.2.

[5] E.J. Christie et al. "Prototyping Strategies: Literature Review and Identification of Critical Variables". In: June 2012. DOI: 10.18260/1-2--21848.

[6] *Color sensor*. URL: https://www.rohm.com/electronics-basics/sensor/color-sensor. (accessed 29.06.2024).

[7] *Color sensor - Working and Applications*. URL: https://www.elprocus.com/color-sensor-working-and-applications/. (accessed 29.06.2024).

[8] Debashi Das. *Interfacing IR sensor module with Arduino*. URL: https://circuitdigest.com/microcontroller-projects/interfacing-ir-sensor-module-with-arduino. (accessed 28.06.2024).

[9] Teach with ICT. URL: https://www.teachwithict.com/hcsr045v.html. (accessed 28.06.2024).

[10] Janis Osis and Uldis Donins. "Topological UML Modeling". In: Elsevier, 2017. Chap. 1.

[11] Alan Moore Sanford Friedenthal and Rick Steiner. “A Practical Guide to SysML”. In: Morgan Kaufmann, 2015. Chap. 9.
[12] Wood magazine stuff. *Kerf bending*. URL: https://www.woodmagazine.com/woodworking-how-to/joinery/kerf-bending. (accessed 28.06.2024).
[13] Transmotec. URL: https://www.transmotec.de/product/755-3262-C/. (accessed 28.06.2024).
[14] Electronics Tutorials. URL: https://www.electronics-tutorials.ws/blog/pulse-width-modulation.html. (accessed 28.06.2024).
[15] Martin Weck. *Requirements Engineering*. URL: https://www.innoq.com/en/articles/2020/03/requirements-engineering/. (accessed 28.06.2024).
[16] Wikipedia. URL: https://en.wikipedia.org/wiki/DC_motor. (accessed 28.06.2024).